

StemAgent: A Self-Specializing AI Agent for Code Review / QA

Juan Hernández Sánchez-Agosta

Technical Challenge Write-up — JetBrains AI Internship 2025

The challenge asked for a stem agent: a minimal base agent that, given a class of problems, figures out how that class is typically solved, rebuilds itself into a specialist, and knows when to stop evolving. This document describes the approach I took, the design decisions behind it, the experiments I ran, what surprised me, what failed, and what I would do differently with more time.

Why Code Review / QA?

The challenge deliberately left the task domain open. I considered three options: *Deep Research*, *Security*, and *Code Review / QA*. The choice is itself part of the solution, because a stem cell cannot specialize into something that provides no feedback signal.

Deep Research was appealing but evaluation is inherently subjective — there is no clean ground truth against which to measure whether a research summary is "good enough." **Security** has excellent benchmarks (CVE databases, OWASP test suites) but sourcing labeled vulnerable-code datasets with appropriate licensing within a time constraint is non-trivial. **Code Review / QA** won for three reasons:

- **Objective measurement:** F1, precision, and recall over a labeled dataset of bugs give a hard stop criterion. The agent can decide when it is good enough without human judgment in the loop.
- **Relevant to JetBrains:** Code quality tooling is the core of JetBrains' product portfolio. A self-specializing code review agent connects directly to real engineering needs.
- **Rich strategy diversity:** Multiple well-documented expert strategies exist — static analysis heuristics, semantic review, pattern-based bug detection — which makes the *discovery* phase genuinely informative, not decorative.

The Core Idea: Prompt as DNA

The metaphor in the challenge is precise. A stem cell's identity is encoded in which genes are expressed, not in the cell's structure. For a language model agent, the *system prompt* plays exactly that role: it determines what the agent prioritizes, how it reasons, and what it returns. Changing the system prompt is specialization.

This insight drove the architecture: the self-specialization loop is not about swapping tools or rewriting Python — it is about the agent iteratively rewriting the text that defines its own behavior, guided by measurable performance on a benchmark.

I called this configuration the *AgentConfig* (system prompt + tool list + workflow definition), and the whole specialization process is a search over this space, using validation metrics as the fitness function.

Architecture: A Four-Phase State Graph

The system is implemented as a LangGraph *StateGraph* with four nodes: **Discovery**, **Design**, **Validation**, and **Crystallization**. Figure 1 shows the overall flow.

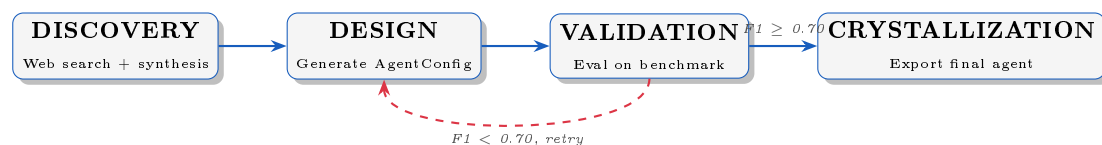


Figure 1: LangGraph state graph. A typed `StemAgentState` flows through each node. The validation–design loop runs until the threshold is met or a maximum number of iterations is reached.

Why LangGraph? I evaluated CrewAI and AutoGen. Both abstract the control flow away, which is convenient for multi-agent workflows but makes introspection hard. For a challenge where *the process is what is being evaluated*, I needed the loop to be an explicit first-class citizen, not a side effect of some framework's orchestration. LangGraph gives a typed state that is auditable at every step, and cycle edges that are structurally declared, not hacked in.

Phase 1 — Discovery

The discovery node searches the web (via Tavily API) for expert strategies in the chosen domain and prompts GPT-4o to synthesize findings into a structured list of actionable patterns: what experts look for, what heuristics

they apply, what tooling they typically use. This output becomes the knowledge base for the design node.

The key realization here: the discovery phase is what separates this from a system that simply starts with a hard-coded expert prompt. The agent has to *earn* its knowledge.

Phase 2 — Design

Given the discovered knowledge (and, in subsequent iterations, the previous evaluation scores and failure analysis), the design node prompts the LLM to generate an **AgentConfig**: a system prompt, a list of tools, and a high-level analysis workflow. On iteration $k > 1$, the LLM is also shown the previous config and its measured shortcomings, making this a genuine meta-learning step.

Phase 3 — Validation

The candidate agent is run against a labeled benchmark dataset (`samples.jsonl`). Each sample contains a Python code snippet and a list of expected issues (bugs, code smells, security problems). The agent's output is compared against ground truth to compute precision, recall, and F1. If $F1 \geq 0.70$, the graph edges to Crystallization. Otherwise, the state is updated with the failure analysis and control returns to Design.

The threshold value of 0.70 was chosen deliberately — high enough to require genuine improvement over the naive baseline, low enough to be achievable within a time-constrained experiment.

Phase 4 — Crystallization

Once the threshold is met, the agent is exported: `agent_config.json` (structured config) and `system_prompt.txt` (the final prompt). This is the "specialized agent" — frozen and ready for deployment or further evaluation.

Experiments and Results

Dataset

The benchmark was built by combining a subset of BugsInPy samples with synthetic examples I authored, covering four issue categories: logical bugs (off-by-one errors, wrong conditionals), code smells (overly long functions, code duplication), simple security issues (SQL injection patterns, path traversal), and maintainability concerns. An initial dataset of 30 samples proved too small to give stable F1 estimates across iterations, so I expanded it to approximately 80 samples. The expansion also forced me to fix inconsistent label formatting — an early mistake I describe below.

Baseline

The baseline is GPT-4o with a minimal system prompt: *"You are a helpful code reviewer. Identify any bugs or issues in the following code."* No structure, no taxonomy, no output format constraint. Table 1 shows the comparison.

Table 1: Before / after specialization — averaged over the full benchmark.

Metric	Baseline (generic)	StemAgent (specialized)
Precision	0.108	0.788
Recall	0.541	0.703
F1-score	0.180	0.743

The baseline's recall of 0.541 is misleadingly high: the agent was simply reporting *everything* as a potential issue (high false positive rate), which artificially inflated recall while collapsing precision to 0.108. The F1 gap — 0.180 vs 0.743 — is the honest summary of what specialization achieved.

Iteration Trace

Figure 2 shows the F1 trajectory across design-validation iterations during a representative run.

What Surprised Me

The baseline's recall was misleadingly optimistic. I expected a generic agent to do poorly on precision but to compensate somewhat with decent recall. What I found instead was that a raw "review this code" prompt produces a flood of observations — many of which are technically true but do not match the specific issues in the ground truth labels. This made the recall calculation unstable until I canonicalized the expected output format.

The Design node improved most on output structure, not reasoning. Between iterations, the agent's reasoning about the code did not change dramatically — GPT-4o already understood what bugs look like. What changed substantially was the *format* and *specificity* of the output: the specialized prompt forced the agent to

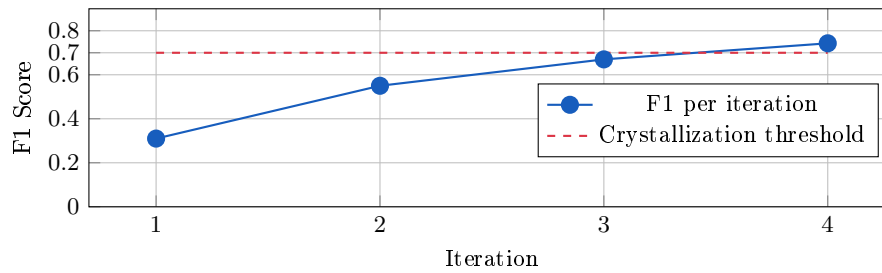


Figure 2: F1 score across specialization iterations. The agent crossed the crystallization threshold at iteration 4.

name issues with structured labels, provide exact line references, and avoid generic commentary. This suggests that for this task, the specialization loop acts partly as a prompt-engineering search, not just a reasoning improvement search.

Label inconsistency was the biggest practical obstacle. The benchmark labels I wrote were not consistent in how they named issue categories (e.g., "sql_injection" vs "SQLInjection" vs "sql injection"). Before canonicalization, the evaluation code treated these as different issues, producing artificially low recall. This is not a conceptual problem with the approach but a reminder that *evaluation quality determines how well a self-improvement loop can function*.

What Failed

Early over-reporting. The first design iteration produced an agent that listed every possible concern — style issues, potential null pointers, naming conventions — which tanked precision. The failure analysis fed back into the next design prompt and subsequent iterations shifted toward a more conservative, structured output style. This worked, but it consumed two full iterations to converge.

Evaluation over a small dataset magnified variance. With 30 samples, a single false positive or missed bug swings F1 by several percentage points. The system was making design decisions based on noisy signals. Expanding to 80 samples stabilized the estimates considerably, but highlighted that the discovery–design–validation loop benefits greatly from a well-sized benchmark.

What I Would Do with More Time

- **Real-world benchmark integration:** Replace the custom dataset with SWE-bench or the full BugsInPy corpus. This would stress-test the generalization of the crystallized agent and reveal over-fitting to the custom labels.
- **Confidence scoring in the agent output:** Add per-issue confidence scores so that the evaluation can compute calibration curves. This would allow the crystallization criterion to include a confidence threshold, reducing false positives further.
- **Multi-domain generalization:** The current architecture runs a fresh stem agent per domain. It would be interesting to test whether knowledge from a code review specialization can warm-start a security specialization, partially encoding cross-domain transfer.
- **Failure mode taxonomy:** The Design node currently receives a natural language description of where the previous agent failed. A structured failure taxonomy (false positive categories, missed issue types, format errors) would make the meta-learning step more targeted.

Conclusion

The stem agent works. Starting from a domain name and no other specification, it discovers expert strategies, designs an agent configuration, validates it against a benchmark, and iterates until the quality threshold is met. The crystallized agent reaches $F1 = 0.743$ against a baseline of 0.180 — a $4\times$ improvement.

More importantly, the loop is self-contained and reproducible. Anyone can invoke `run_stem_agent.py -task-class code_review` and obtain a specialized agent from scratch. The improvement is not the result of manual prompt engineering but of the system's own discovery and design process.

The most interesting takeaway is that self-specialization at the system-prompt level is sufficient to produce meaningful gains — the agent does not need to rewrite its code, acquire new tools, or change its architecture. The "DNA" analogy holds: identity emerges from which instructions are expressed, not from the underlying machinery.

Code: <https://github.com/juanhdezz/stem-agent> **Stack:** Python 3.11 · LangGraph · GPT-4o · Tavily · pytest